Daily Grid: Basic Site Setup Plan

Oct 20, 2026 · @Alex

Goal and scope

The basic site is done when a public URL shows a Daily Grid homepage, every pull request runs tests and gets a preview link, and `main` only changes through a passing PR. That last part matters most: it is the safety net the agent will work inside.

In scope for you to build by hand (with Claude Code helping):

- A static homepage with today's date and a "first puzzle coming soon" placeholder
- One small, tested utility (the daily seed) so CI has real code to check
- Linting, type checks, unit tests with coverage, and one end-to-end smoke test
- CI on GitHub Actions, hosting with automatic PR previews, and branch protection
- A `CLAUDE.md` and an issue template, so the agent has rules and well-formed tasks from day one

Left for the agent: every game, the share button, stats, the archive, and the "How this site is built" page.

Stack

Every choice below favours a small, conventional, well-documented setup, because agents do their best work in codebases that look like the thousands they have already seen.

Layer	Choice	Why
Framework	Astro, static output	Plain HTML by default, add interactive islands (React, Svelte or plain TS) per game later
Language	TypeScript, strict mode	Type errors become a CI gate the agent must pass
Unit tests	Vitest + `@vitest/coverage-v8`	Fast, works with Astro's Vite config, produces coverage reports
End-to-end tests	Playwright (Chromium only to start)	Proves the page actually renders in a browser

Layer	Choice	Why
Lint and format	ESLint + Prettier	Keeps agent diffs consistent with yours
CI	GitHub Actions	Same tool your agent will run on
Hosting	Cloudflare Pages	Free tier, automatic preview URL on every PR
Package manager	npm	Nothing extra to install in CI or for the agent

Netlify or Vercel work equally well for hosting if you prefer them; the only must-have is a preview deploy per PR.

Prerequisites

- [ ] Node.js, current LTS release, installed locally
- [ ] Git and a GitHub account (the repo will be public)
- [ ] A Cloudflare account (free plan)
- [ ] Claude Code installed and signed in
- [ ] An Anthropic API key with a monthly spend limit set; not needed until the agent step, but worth setting up now

Setup steps

Work through these in order on a branch, then merge with a PR once CI exists. Expect one to two evenings.

1. Create the repo and scaffold Astro

Create a public GitHub repo named `daily-grid`, then:

```
npm create astro@latest daily-grid -- --template minimal
cd daily-grid
npx astro add react   # optional now; useful once games need interactivity
```

Pick TypeScript **strict** when prompted (`tsconfig.json` should extend `astro/tsconfigs/strict`).

2. Lay out the folders for growth

```
src/
  layouts/BaseLayout.astro    # <html>, meta tags, header, footer
  pages/index.astro           # homepage
  lib/daily-seed.ts           # shared utilities
  lib/daily-seed.test.ts      # tests sit next to the code
  games/README.md             # one folder per game, added later by the agent
e2e/home.spec.ts              # Playwright tests
.github/workflows/ci.yml
.github/ISSUE_TEMPLATE/feature.yml
CLAUDE.md
```

The convention to set now: each game lives in `src/games/<name>/` with its logic in plain TypeScript, its tests beside it, and its UI component separate. Pure logic is what makes high coverage easy.

3. Build the homepage

`BaseLayout.astro` holds the page shell, a header with the site name, and a footer linking to the GitHub repo. `index.astro` shows the title, today's date, and a "Today's puzzle is coming soon" card.

One gotcha: the site is static, so a date written at build time goes stale. Render the date with a small inline script in the browser instead. Like Wordle, the puzzle follows each visitor's local date and changes at their local midnight, which is what dateKey below already does.

4. Add the daily seed utility

This gives CI real code to test, and every future game will use it to make the same puzzle for everyone on a given day.

```
// src/lib/daily-seed.ts
export function dateKey(date: Date): string {
  const y = date.getFullYear();
  const m = String(date.getMonth() + 1).padStart(2, '0');
  const d = String(date.getDate()).padStart(2, '0');
  return `${y}-${m}-${d}`;
}

// FNV-1a: turns a string into a 32-bit seed
export function seedFromKey(key: string): number {
  let h = 0x811c9dc5;
  for (let i = 0; i < key.length; i++) {
    h ^= key.charCodeAt(i);
```

```
    h = Math.imul(h, 0x01000193);
  }
  return h >>> 0;
}

// mulberry32: small, fast, deterministic PRNG returning [0, 1)
export function createRng(seed: number): () => number {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6d2b79f5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

Tests should cover: `dateKey` pads single digits, the same key always gives the same seed, different keys give different seeds, the same seed always gives the same sequence, and every value is in [0, 1).

5. Add the tooling

```
npm i -D vitest @vitest/coverage-v8 @playwright/test @astrojs/check
typescript \
  eslint typescript-eslint eslint-plugin-astro prettier prettier-plugin-astro
```

`package.json` scripts:

```
"dev": "astro dev",
"build": "astro build",
"preview": "astro preview",
"check": "astro check",
"lint": "eslint . && prettier --check .",
"test": "vitest run",
"test:coverage": "vitest run --coverage",
"test:e2e": "playwright test"
```

`vitest.config.ts` uses Astro's Vite config, limits tests to `src/`, and sets a coverage floor:

```
import { getViteConfig } from 'astro/config';

export default getViteConfig({
  test: {
    include: ['src/**/*.test.ts'],
    coverage: {
      provider: 'v8',
      include: ['src/lib/**', 'src/games/**'],
      reporter: ['text', 'json-summary', 'html'],
      thresholds: { lines: 90, functions: 90, branches: 85 },
    },
  },
});
```

`playwright.config.ts` sets `testDir: 'e2e'`, Chromium only, and a `webServer` that runs `npm run build && npm run preview` on `http://localhost:4321`. The one smoke test checks that the homepage loads, shows the title, and shows a date.

6. Add CI

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: lts/*
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run check
      - run: npm run test:coverage
      - run: npx playwright install --with-deps chromium
      - run: npm run test:e2e
      - uses: actions/upload-artifact@v4
```

```
        if: always()
        with:
          name: coverage
          path: coverage/
```

The uploaded `coverage/coverage-summary.json` is what the future dashboard will chart.

7. Connect hosting

In Cloudflare, go to Workers & Pages, create a Pages project, and connect the GitHub repo. Use the Astro preset, build command `npm run build`, and output directory `dist`. Production deploys from `main`; every other branch, and so every PR, gets its own preview URL automatically.

8. Protect `main`

In the repo's Settings, add a branch ruleset for `main`: require a pull request, require the `verify` status check to pass, and block force pushes and deletions. Set required approvals to 0 for now, since GitHub won't let you approve your own PRs; you can revisit this once the agent is opening PRs under a bot identity.

Get it ready for the agent

Two files make the agent's first run go well: a `CLAUDE.md` that tells it how this repo works, and an issue form that forces every task into the same clear shape.

CLAUDE.md

```
# Daily Grid

A static site of daily puzzle games. Everyone gets the same puzzle each day,
generated from a date-based seed. No backend; per-visitor data uses
localStorage.

## Commands
- npm run dev / build / preview
- npm run lint, npm run check, npm run test:coverage, npm run test:e2e
- All four checks must pass before you open a PR.

## Structure
- src/lib/        shared utilities (daily-seed.ts: dateKey, seedFromKey,
createRng)
- src/games/<name>/  one folder per game: logic.ts, logic.test.ts, UI
component
```

```
- src/pages/      one route per page; games live at /games/<name>
- e2e/            Playwright tests

## Rules
- Game logic is pure TypeScript with no DOM access, so it can be unit tested.
- All randomness goes through createRng(seedFromKey(dateKey(date))). Never
Math.random().
- Every new function gets unit tests. Every new page gets an e2e smoke test.
- Keep coverage at or above the thresholds in vitest.config.ts. Never lower
them.
- Accessible by default: keyboard playable, labelled controls, no colour-only
meaning.
- Do not edit anything in .github/, CLAUDE.md, or vitest.config.ts
thresholds.
- Do not add dependencies unless the issue allows it; say why in the PR if
you do.
- Keep PRs to the scope of the issue. Note follow-up ideas in the PR
description.

## Pull requests
- Title: the issue title. Body: "Closes #<n>", a summary, how it was tested,
  and anything you were unsure about.
```

Issue form

```
# .github/ISSUE_TEMPLATE/feature.yml
name: Feature
description: A task for a human or the agent
title: "[Feature] "
body:
  - type: textarea
    id: goal
    attributes:
      label: Goal
      description: What should exist when this is done, in one or two
sentences.
    validations:
      required: true
  - type: textarea
    id: acceptance
    attributes:
      label: Acceptance criteria
      description: A checklist the PR can be judged against.
      placeholder: "- [ ] ..."
    validations:
      required: true
```

```
  - type: textarea
    id: notes
    attributes:
      label: Notes and constraints
      description: Files to touch, things to avoid, dependencies allowed.
```

Definition of done for any PR

- Lint, type check, unit tests with coverage, and e2e tests all pass in CI
- The preview URL shows the change working on desktop and a phone-width screen
- The PR links its issue and every acceptance criterion is met
- No coverage threshold, workflow file, or `CLAUDE.md` rule was changed by the agent

Checklist and next step

- [ ] Public repo created and Astro scaffolded in strict TypeScript
- [ ] Homepage renders with a client-side date and a placeholder card
- [ ] `daily-seed.ts` written with passing unit tests above the coverage floor
- [ ] Lint, type check, Vitest, and Playwright all run locally
- [ ] CI workflow passes on a PR
- [ ] Cloudflare Pages deploys `main` and posts a preview URL on PRs
- [ ] Ruleset on `main` requires a PR and a passing `verify` check
- [ ] `CLAUDE.md` and the feature issue form committed
- [ ] First real issue written and waiting: "Add a daily Lights Out game"

Next comes the agent: a workflow that runs Claude Code when you add an `agent` label to an issue, has it implement the change against `CLAUDE.md`, and opens a PR that this CI and preview setup then checks. Its first job can be that Lights Out issue.